**init.php Function List:**
session_start(): Initializes/Resumes the user session.
include_db_config(): Connects to the database.
define_constants(): Stores game math (Gold penalties, weight limits).
check_auth_status(): The "Gatekeeper" redirecting unauthorized users.
handle_error_reporting(): Toggles visibility of system errors.
sanitize_input(): The security filter for all user-provided text.
sync_player_session_data(): Refreshes the session with live database values.

init.php: Detailed Function Analysis

**1. session_start()**
Input: None. (Uses the browser's session cookie).
Output: Boolean. (Returns true if session is active).
Persistent Variables Created:
$_SESSION['userId'] (Int): Unique ID from the Users table.
$_SESSION['username'] (String): The ownerId used for character linking.
$_SESSION['characterId'] (Int): The ID of the currently selected character from the Characters table.

**2. include_db_config()**
Input: None.
Output: Object (The MySQLi connection resource).
Global Variable:
$conn (Resource/Object): This variable is now available to all other files to execute SQL queries.

**3. define_constants()**
Input: None.
Output: Void (Sets global constants).
Key Constants Defined:
DEATH_GOLD_PENALTY (Float): 0.10 (used in reset_game.php).
STR_TO_WEIGHT_CAP (Int): 5 (used in process_item.php).
MIN_ACCURACY (Float): 0.75 (used in process_combat.php).

**4. check_auth_status()**
Input: None. (Checks the $_SESSION state).
Output: Void (Executes a Header Redirect if check fails).
Logic: If $_SESSION['userId'] is null, it sends the browser to index.php.

**5. handle_error_reporting()**
Input: $isDevMode (Boolean).
Output: Void.
Logic: If true, it sets PHP's display_errors to 1. If false, it sets it to 0.
Variable: $isDevMode is usually hardcoded at the top of init.php.

**6. sanitize_input()**
Input: $rawData (String or Array).
Output: $cleanData (String or Array).
Logic: Uses mysqli_real_escape_string() on the input string to prevent SQL injection. It ensures that characters like ' or " don't break your database queries.

**7. sync_player_session_data()**
Input: $characterId (Int).
Output: Boolean (True if sync succeeded).
Internal Variables:
$sql (String): The query SELECT * FROM Characters WHERE id = $characterId.
$result (Object): The database response.
Persistent Variable Updates: Refreshes $_SESSION['hp'], $_SESSION['gold'], etc., with the latest values from MySQL.

---

**db_config.php Structure**
Credentials (Variables):
$db_host (String): Usually "localhost".
$db_user (String): Your database username.
$db_pass (String): Your database password.
$db_name (String): "lingua_quest_db".
mysqli_connect(): Opens the connection using the variables above.
check_connection(): Validates the link.
mysqli_set_charset(): (New/Essential) Forces the connection to use utf8mb4.
Why: Even if your table is utf8mb4, if the connection isn't told to use it, your French accents (é, à) will show up as weird symbols like ``.

Detailed Variable Analysis

**1. mysqli_connect()**
Inputs: $db_host, $db_user, $db_pass, $db_name (All Strings).
Output: $conn (An Object or Resource).
Description: This is the "Key" that stays in memory. Every other file uses this $conn object to talk to MySQL.

**2. check_connection()**
Input: $conn (Object).
Output: Boolean or Die.
Description: If $conn is false, it stops the script and displays an error.

**3. mysqli_set_charset()**
Input: $conn (Object), "utf8mb4" (String).
Output: Boolean.
Description: Ensures that the French text travels from the DB to the PHP file without getting corrupted.

---

**auth_worker.php:** Detailed Function Analysis

**1. login_user()**
Purpose: Authenticates an existing user against the Users table.
Inputs:
$username (String): Captured from the login form.
$password (String): The plain-text password entered by the user.
Outputs: Boolean. (Returns true if credentials match).

Logic:
Fetches passwordHash from the database.
Uses PHP's password_verify($password, $hash) to compare the input to the hash.
If successful, assigns $_SESSION['userId'] and $_SESSION['username'].

**2. register_user()**
Purpose: Creates a new account in the Users table.
Inputs:
$username (String): Unique identifier.
$email (String): Validated email format.
$password (String): Plain-text password.
Outputs: Int (Returns the new userId) or Boolean (false if failure).
Logic:
Uses password_hash() to encrypt the password before storage.
Executes an INSERT into the Users table.

**3. validate_unique_credentials()**
Purpose: Ensures no duplicate usernames or emails exist before registration.
Inputs:
$username (String)
$email (String)
Outputs: Boolean. (true if the data is unique and safe to use).
Logic: Performs a SELECT check. If a row is found, it returns false to trigger a UI warning.

**4. create_character()**
Purpose: Initializes a new hero and links it to the User.
Inputs:
$ownerId (String): Taken from $_SESSION['username'].
$characterName (String): The name chosen in the creation UI.
Outputs: Int. (The new characterId).
Logic:
Inserts a row into the Characters table with default stats (HP: 100, STR: 10, etc.).
Sets the starting currentLocationID (e.g., 101).

**5. fetch_user_characters()**
Purpose: Retrieves all characters associated with the logged-in account for selection.
Inputs:
$ownerId (String).
Outputs: Array/JSON. (A list of characters with basic stats).
Logic: SELECT * FROM Characters WHERE ownerId = $ownerId. This allows the user to choose which character to "Enter World" with.

**6. select_character()**
Purpose: Sets the active character for the current session.
Inputs:
$characterId (Int).
Outputs: Void (Redirects to adventure.php).
Logic: Assigns $characterId to $_SESSION['characterId'], which locks the engine into that specific character's state.

Engineering Alignment for the "Bible"

Data Consistency: Note that $ownerId is a String (Username), but $userId and $characterId are Integers.
Security Protocol: This file should always be the target of a POST request, never a GET, to keep the password data out of the browser's history logs.

---

**engine.js:** Detailed Function Analysis

For engine.js, we are defining the "Brain" of the frontend. As a developer, I treat this file as the State Manager. It doesn't know the "Truth" of the database; it simply asks the PHP "Judges" for information and tells the UI how to react.
Here is the detailed function list for engine.js with inputs, outputs, and variable types:

**1. initGame()**
Purpose: The primary bootloader. Orchestrates the initial load of the player and the room.
Inputs: None.
Outputs: Void.
Logic:
Calls fetchPlayerStats() and fetchRoomData() simultaneously.
Initializes the speech.js listeners.
Variable: isGameActive (Boolean), set to true upon success.

**2. sendAjaxRequest()**
Purpose: The master communication wrapper for all server-side calls.
Inputs:
endpoint (String): The target PHP file (e.g., "move_player.php").
payload (Object): The data to send (e.g., { direction: 'n' }).
Outputs: Promise (Returns a JSON Object).
Logic: Uses the fetch() API with the POST method. It centralizes error handling so a server crash doesn't break the JS loop.

**3. parseInput()**
Purpose: Evaluates player commands to determine which logic worker to trigger.
Inputs:
rawInput (String): The text from STT or the keyboard.
Outputs: Void.
Logic:
Checks for keywords (e.g., "Nord," "Prendre," "Attaque").
Variable: commandType (String), used to route the input to movePlayer(), processItem(), or combat.js.

**4. movePlayer()**
Purpose: Logic for navigating the 8-point compass.
Inputs:
direction (String): The compass coordinate (n, s, e, w, u, d, in, out).
Outputs: Void.
Logic:
Sends the direction to move_player.php.
If the response contains checkpointReached: true, it triggers the ui.js notification.
Upon success, calls fetchRoomData().

**5. fetchRoomData()**
Purpose: Requests the bilingual text and exits for the current node.
Inputs:
nodeId (Int).
Outputs: Void.
Logic:
Calls get_room.php.
Passes the resulting JSON to ui.renderRoomText() and ui.updateCompass().

**6. fetchPlayerStats()**
Purpose: Syncs the local UI with the database "Truth" regarding HP, Gold, and XP.
Inputs: None. (Uses the character ID stored in the PHP session).
Outputs: Void.
Logic:
Calls get_player.php.
Passes the data to ui.refreshStatsBar().

**7. handleGameStateChange()**
Purpose: Listens for "Global" events like death or level-ups.
Inputs:
stateUpdate (JSON Object).
Outputs: Void.
Logic:
If hp <= 0, it triggers a redirect to the reset_game.php sequence.
If levelUp is true, it triggers a UI celebration effect.

Engineering Alignment for the "Bible"
Asynchronous Nature: All these functions are Asynchronous, meaning they wait for the server without freezing the player's screen.
Data Types: JavaScript is "loosely typed," but for safety, we must ensure that nodeId is always treated as a Number and direction is always a lowercase String.

---

**speech.js**: The Dedicated "Ears" (Input Only)

**1. initSpeech()**
Description: Checks if the browser is compatible with the webkitSpeechRecognition API and initializes the object.
Input: None.
Output: Boolean (Returns true if initialized, false if the browser is unsupported).
Internal Variable: recognition (Object): The global instance of the listener.

**2. startListening()**
Description: Triggers the microphone. It sets the linguistic context specifically to French to ensure high accuracy for your learning goals.
Input: None.
Output: Void.
Internal Variables:
recognition.lang (String): Fixed to 'fr-FR'.
recognition.interimResults (Bool): Set to false to wait for a full phrase.

**3. recognition.onresult (The Dispatcher Event)**
Description: The most critical point in the file. It captures the raw audio, converts it to text, and hands it off to the Engine.
Input: event (SpeechRecognitionEvent).
Output: Void (Dispatches data to engine.routeVocalCommand).
Data Flow:
Extracts transcript from the event.
heardText (String): Normalizes the string (lowercase, trimmed).
Calls engine.routeVocalCommand(heardText).

**4. handleRecognitionError(event)**
Description: Manages specific API errors like "no-speech" (silence) or "audio-capture" (mic blocked).
Input: event (Object).
Output: Int (Updates the persistent failCount).
Logic: Tracks consecutive fails. When failCount >= 3, it signals ui.js to show the manual keyboard input.

**5. stopListening()**
Description: Immediately cuts the power to the microphone.
Input: None.
Output: Void.
Usage: Used for timing out turns in combat or manual cancellations.

**6. getSimilarity(s1, s2)**
Description: The internal grading tool. It calculates the Levenshtein distance to provide the Similarity Score.
Input:
s1 (String): The player's vocal input.
s2 (String): The target phrase from the database.
Output: Float (A decimal between 0.0 and 1.0).
Logic: (LongerLength - EditDistance) / LongerLength.

---

**ui.js:** Function Analysis (Integrated Version)

**1. showTab(tabId)** (Tabbed Interface)
Description: Manages the visibility of the "Basics," "Stats," and "Inventory" panels.
Input: tabId (String).
Logic: Loops through a defined array of IDs, setting all to display: none except the active one.
Data Flow: If tabId === 'tabInv', it triggers a call to get_inventory.php to ensure the list is fresh.

**2. toggleTranslation()** (Bilingual Control)
Description: Flips the global isEnglishMode flag and immediately redraws the room text.
Input: None.
Output: Void.
Bible Alignment: Ensures your "Bilingual Content" goal is met without page reloads.

**3. updateRoomDisplay(data)** (The Master Painter)
Description: Clears the story box and injects the new description, clickable NPCs, and floor items.
Input: data (JSON Object from get_room.php).

Refinement: Instead of raw HTML buttons, we will use your .btn-neon and .btn-outline classes to keep the "Style" consistent.
Logic: Uses .replace(/\n/g, '<br>') for formatting.

**4. updateCompass(room)** (Navigation Controller)
Description: Disables invalid paths and enables valid ones.
Input: room (JSON Object).
Engineering Detail: Uses btn.style.opacity for visual feedback. In our new version, we will move this to a CSS class like .btn-disabled to keep the logic out of the JS.

**5. handleManualSubmit()** (Keyboard Fallback)
Description: The "Secondary Interface" promised in your Bible.
Input: typed (String from an input field).
Logic: Converts "Nord" or "Est" into a changeRoom() call. It resets the failCount upon success.

**6. setCombatUI(isActive, enemyData)** (Arena Transition)
Description: Transforms the Dialogue Overlay into the "Combat Overlay."
Input: isActive (Bool), enemyData (JSON Object).
Logic: Toggles .combat-mode class on the overlay and activates the HP bar containers.

**7. updateHPBars(eHP, eMax, pHP, pMax)** (Combat Feedback)
Description: Visualizes damage in real-time during battle.
Input: Current and Max HP for both Player and NPC (Integers).
Variables: ePercent and pPercent (Floats) to set the bar widths.

**8. confirmReset()** (The World Reset)
Description: A safety-gated trigger for reset_game.php.
Input: None.
Logic: Uses window.confirm(). Upon success, it calls location.reload() to re-sync the entire game state.

**9. speakText(phrase)** (The "Mouth")
Description: (As discussed) Reads the description text aloud using French TTS.

Software Engineer's Compatibility Audit
Global Variables Required: For this file to work, engine.js must maintain currentRoomData, isEnglishMode, and playerStats.
Naming Conventions: Your old code used room.northTarget. In our new MySQL schema, we named it northTarget. These match perfectly.
Style Fix: To make this look better, we will stop using style.background = "#1565c0" in the JS. Instead, we will use element.classList.add('active-cyan'), keeping all the "Vibe" control in your CSS files.

---

**get_room.php**: The World Fetcher

this file as a "Read-Only Service" that must gather data from three different tables to provide the "Truth" about the player's immediate surroundings.

**1. fetch_node_data($nodeId)**
Description: The primary SQL engine. It retrieves the core room attributes, bilingual text, and compass targets.
Input: $nodeId (Int): Captured from the current session or a move request.
Output: Array (Raw row data from Locations).
SQL Logic: SELECT * FROM Locations WHERE nodeId = ?.
Data Types: nodeId (Int), textFrench (Text), northTarget (Int), etc.

**2. check_room_state($characterId, $nodeId)**
Description: Checks if the player has visited this room before to handle the isDiscovered logic.
Input: $characterId (Int), $nodeId (Int).
Output: Boolean.
SQL Logic: SELECT isDiscovered FROM Character_Room_State WHERE characterId = ? AND nodeId = ?.
Verdict: This ensures the "Fog of War" on the map is character-specific, not global.

**3. get_present_npcs($nodeId, $characterId)**
Description: Finds all NPCs currently located in this room specifically for this character.
Input: $nodeId (Int), $characterId (Int).
Output: JSON Array of NPC objects.
SQL Logic: A JOIN between Npcs (Definitions) and Character_NPC_State (Location).
SELECT n.npcId, n.npcNameFrench, s.currentHitPoints, s.isDead ...
Bible Alignment: This is what provides the data for your "Personnages ici" buttons in ui.js.

**4. get_ground_items($nodeId)**
Description: Identifies any items where the ownerType is 'Room' and ownerId is the current node.
Input: $nodeId (Int).
Output: JSON Array of Item objects.
SQL Logic: A JOIN between ItemInstances and ItemLibrary.
Bible Alignment: This provides the list for your "Objets au sol" display.

**5. generate_room_json($room, $npcs, $items, $discovered)**
Description: The "Plating" function. It bundles all the gathered arrays into one clean JSON object to send to engine.js.
Input: Multiple Arrays.
Output: JSON String.
Logic: Uses json_encode() to ensure the data is browser-readable.

---

**move_player.php**: The Navigation Judge

**1. validate_move_request($charId, $direction)**
Description: Checks the current room to see if the requested direction actually leads somewhere.
Input:
$charId (Int): From session.
$direction (String): The compass point (n, s, e, w, u, d, in, out).
Output: Int (The targetNodeId) or Boolean (False if blocked).
SQL Logic: SELECT northTarget, southTarget... FROM Locations WHERE nodeId = (SELECT currentLocationID FROM Characters WHERE id = ?).
Verdict: This prevents "teleporting" or walking through walls if a player tries to hack a request.

**2. update_player_location($charId, $targetNodeId)**
Description: The primary database update.
Input: $charId (Int), $targetNodeId (Int).
Output: Boolean (True on success).
SQL Logic: UPDATE Characters SET currentLocationID = ? WHERE id = ?.

**3. process_auto_save($charId, $targetNodeId)**
Description: Handles the Auto-Save/Checkpoint logic we designed for the Project Bible.
Input: $charId (Int), $targetNodeId (Int).
Output: Boolean (True if a new checkpoint was registered).
Logic:
Fetches isCheckpoint and nodeId from Locations for the target room.
Fetches respawnNodeId from Characters.
High-Water Mark Check: IF (target.isCheckpoint == 1 AND target.nodeId > character.respawnNodeId).
Action: UPDATE Characters SET respawnNodeId = ? WHERE id = ?.

**4. register_room_discovery($charId, $targetNodeId)**
Description: Marks the room as seen so the minimap can update.
Input: $charId (Int), $targetNodeId (Int).
Output: Void.
SQL Logic: INSERT IGNORE INTO Character_Room_State (characterId, nodeId, isDiscovered) VALUES (?, ?, 1).

**5. generate_move_response($status, $targetId, $checkpointSaved)**
Description: Packages the result for engine.js.
Input: $status (String), $targetId (Int), $checkpointSaved (Bool).
Output: JSON String.
Example: {"status": "success", "newNodeId": 102, "checkpoint": true}.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$charId
Integer
init.php
The session-validated player ID.
$direction
String
POST request
The intended move (e.g., 'n').
$targetNodeId
Integer
Locations table
The ID the player is moving into.
$respawnNodeId
Integer
Characters table

The current "Safe Point" address.
$isCheckpoint
TINYINT
Locations table
Flag used to trigger the save logic.

Engineering Alignment for the "Bible"
The "Slide" Safety: By checking target.nodeId > character.respawnNodeId, this file ensures that sliding down a hill (backtracking) never overwrites a more advanced checkpoint.
Atomic Operations: This file should update the location and the checkpoint in a single script execution to ensure the player's state is always "in sync."
Triggering UI: The checkpoint boolean in the JSON output is what tells ui.js to flash the "Progress Registered" neon notification.

---

**get_inventory.php**: The Ownership Fetcher

**1. fetch_inventory_list($characterId)**
Description: The primary SQL engine that performs a JOIN between the live instances and the static definitions.
Input: $characterId (Int): Pulled from the $_SESSION.
Output: Array of objects (The combined data of the Library and Instance).
SQL Logic:sql

```
SELECT lib.nameFrench, lib.nameEnglish, lib.itemType, lib.weight, lib.extraData, inst.instanceId
FROM ItemInstances inst
JOIN ItemLibrary lib ON inst.itemId = lib.itemId
WHERE inst.ownerType = 'Player' AND inst.ownerId = ?;
```

Verdict: This is where the "Stateless" name meets the "Stateful" owner.

**2. calculate_total_weight($inventoryArray)**
Description: Sums up the weight of all items currently in the player's possession.
Input: $inventoryArray (Array).
Output: Decimal/Float (Total Weight).
Logic: Loops through the results of the SQL query and adds up the weight column.

**3. get_encumbrance_limit($characterId)**
Description: Calculates the player's maximum carrying capacity based on their Strength.
Input: $characterId (Int).
Output: Integer (Max Weight).
Logic: Fetches strength from the Characters table and multiplies it by the STR_TO_WEIGHT_CAP (5) constant defined in init.php.

**4. package_inventory_json($items, $totalWeight, $maxWeight)**
Description: Bundles the list, the current weight, and the max capacity into a single JSON response.
Input: Array, Float, Int.
Output: JSON String.
Example: {"items": [...], "totalWeight": 12.5, "maxWeight": 50}.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$characterId
Integer
init.php
The primary key of the player.
$instanceId
Integer
ItemInstances
The unique ID of the specific object (e.g., Bob's sword).
$weight
Decimal/Float
ItemLibrary
Used to calculate if the player is too heavy.
$extraData
JSON Object
ItemLibrary
Contains "DNA" like healAmount or durability.
$ownerType
ENUM
ItemInstances
Must be set to 'Player' for this fetcher.

Engineering Interaction Audit
Which tables does it READ from?
ItemInstances: To find out what the player actually owns.
ItemLibrary: To get the names and weights of those items.
Characters: To get the strength needed for the encumbrance limit.
Which tables does it WRITE to?
None. This is a "Read-Only" fetcher. It never changes the database.
Interaction with WorldTemplates:
None. Like the movement logic, it only cares about the "Live" ItemInstances table.

Software Engineer's "Bible" Note:
By including the maxWeight in the JSON, the ui.js can display a "Weight: 12.5 / 50" bar. More importantly, it allows engine.js to block the player from picking up a new item before it even sends a request to the server, making the game feel responsive.

---

**get_dialogue.php**: The Linguistic Bridge

**1. fetch_npc_context($npcId, $characterId)**
Description: Retrieves the NPC's current identity and the player's specific progress in their conversation.
Input: $npcId (Int), $characterId (Int).
Output: Array (Raw status data).
SQL Logic:sql

```
SELECT n.dialogueTreeId, s.currentDialogueNode
FROM Npcs n
JOIN Character_NPC_State s ON n.npcId = s.npcId
WHERE n.npcId = ? AND s.characterId = ?;
```

Verdict: This ensures the NPC remembers that you already said "Bonjour" and are now on the "Next" part of the story.

**2. load_dialogue_tree($treeId)**
Description: The "JSON Reader." It locates and parses the specific .json file containing the branching logic for the conversation.
Input: $treeId (String).
Output: JSON Object (The full tree).
Logic: Uses file_get_contents() and json_decode() to turn the external file into a PHP-readable structure.

**3. extract_current_node($tree, $nodeKey)**
Description: Isolates the specific "Node" (the single screen of text and options) that the player should see right now.
Input: $tree (Object), $nodeKey (String).
Output: Array (The active node).
Logic: Traverses the JSON object to find the entry matching the currentDialogueNode (e.g., "GREETING_01").

**4. prepare_vocal_options($node)**
Description: The "STT Target Preparer." It extracts the French response options and prepares the "Target Phrases" for your speech.js to listen for.
Input: $node (Array).
Output: JSON Array of options.
Logic: It strips punctuation from the labels to create a "clean" phrase for easier voice recognition (e.g., "Oui, s'il vous plaît!" becomes "oui s'il vous plaît").

**5. generate_dialogue_json($npcName, $nodeContent)**
Description: The "Plating" function. It bundles the NPC's name, the text, and the player's options into the final package for the UI.
Input: String, Array.
Output: JSON String.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$npcId
Integer
engine.js
The ID of the person the player clicked.
$dialogueTreeId
String
Npcs Table
The filename of the JSON tree (e.g. village_elder).
$currentNode
String

NPC_State
The "Current ID" in the tree (e.g. NODE_05).
$textFrench
String/Text
JSON File
The actual NPC dialogue to be displayed/spoken.
$options
Array
JSON File
The list of buttons/phrases the player can choose.

Engineering Interaction Audit
Which tables does it READ from?
Npcs: To find the dialogueTreeId blueprint.
Character_NPC_State: To find the player's current currentDialogueNode.
Which files does it READ from?
JSON Content Files: It fetches the actual text and branching logic from your external story files.
Which tables does it WRITE to?
None. This is a "Fetcher" only. The "Writing" (updating the node) happens in process_command.php after the player speaks.

Software Engineer's "Bible" Note
By storing the complex branching logic in JSON files instead of the Database, you follow the "Decoupled Content" rule. You can write long, complex stories with 50 branches in a text editor without ever needing to add a new column to your MySQL tables.

---

**get_player.php**: The Identity Fetcher

**1. fetch_character_core($characterId)**
Description: The primary SQL engine that retrieves the character's base stats and current standing.
Input: $characterId (Int): Pulled from the $_SESSION.
Output: Array (Raw row data from the Characters table).
SQL Logic: SELECT * FROM Characters WHERE id = ?.
Verdict: This provides the raw numbers (HP, STR, AGI, etc.) that define the player's power.

**2. calculate_total_armor($characterId)**
Description: Scans the player's equipped items to determine their total damage reduction.
Input: $characterId (Int).
Output: Integer (Total Armor Value).
SQL Logic: SELECT SUM(JSON_EXTRACT(lib.extraData, '$.armorValue')) FROM ItemInstances inst JOIN ItemLibrary lib ON inst.itemId = lib.itemId WHERE inst.ownerId = ? AND inst.ownerType = 'Player' AND JSON_EXTRACT(lib.extraData, '$.isEquipped') = true.
Bible Connection: Fulfills the "Armor absorbing damage" requirement in the Project Bible.

**3. get_learning_progress($characterId)**
Description: Specifically pulls the speechSuccessCount to update the player on their linguistic mastery.
Input: $characterId (Int).
Output: Integer.
Bible Connection: Directly supports the Reward Economy and the "Learning Mechanics" section.

**4. package_player_json($core, $armor, $learning)**
Description: The "Plating" function. Bundles the stats, learning progress, and calculated armor into a single JSON object.
Input: Array, Int, Int.
Output: JSON String.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$characterId
Integer
init.php
The session-validated key for the player.
$hitPoints
Integer
Characters
Current health for the UI HP bar.
$strength
Integer
Characters
Used for both damage math and weight capacity.
$gold
Decimal/Float
Characters
The currency for the reward economy.
$ownerId
String
Characters
The link back to the Users account table.

Engineering Interaction Audit
Which tables does it READ from?
Characters: For the core RPG attributes and location.
ItemInstances / ItemLibrary: To calculate the current armor benefits from held/equipped items.
Which tables does it WRITE to?
None. This is a "Read-Only" service.
Interaction with Global Utility (init.php):
This file relies on init.php to provide the $characterId. If the session is missing, this file will return an "Unauthorized" error instead of player data.

Software Engineer's "Bible" Note:
This fetcher is the bridge for the Combat Overlay. When a battle starts, engine.js calls this file to ensure it has the most up-to-date hitPoints and armor values before displaying the "Arena" view. By keeping the calculation logic in PHP, we ensure the player cannot "spoof" their HP in the browser.

---

**combat.js**: The Arena Manager (Revised)

**1. startCombat(npcData)**
Description: Freezes exploration and initializes the "Battle State."
Input: npcData (JSON Object) from get_room.php.
Output: Void.
Variables:
playerParty (Array): Filters currentRoomData.npcs to find active allies (isAlly == 1 && isDead == 0).
isCombatActive (Boolean): Set to true.
Logic: Triggers the UI swap and notifies the backend that a combat session has begun for the current character.

**2. executeCombatTurn(heardText)**
Description: The "Dispatcher" for voice commands during battle.
Input: heardText (String): Normalized text from speech.js.
Output: Void.
Logic:
Compares heardText against "J'attaque" or "Je m'enfuis".
Calls speech.js -> getSimilarity() to get the actual performance score.
Passes that score to submitCombatAction().

**3. submitCombatAction(type, score)**
Description: Sends the player's performance to the server "Judge."
Input: type (String: 'ATTACK' | 'FLEE'), score (Float).
Output: Promise/JSON.
Logic: Sends a request to process_combat.php. This file now calculates damage for the player and all active allies in one go.

**4. handleCombatResult(response)**
Description: Processes the "Judge's" decision and updates the visual arena.
Input: response (JSON Object) from the server.
Logic:
Updates health bars via ui.updateHPBars().
Prints the battle log (e.g., "Parfait ! You hit for 15. Your ally hits for 5!").
Decision Gate: If response.npcHp <= 0, it triggers initiateVictorySequence(). If playerHp <= 0, it triggers initiateDefeat().

**5. initiateVictorySequence(victoryData)**
Description: Handles the transition from a kill to the loot screen.
Input: victoryData (JSON Object) containing attribute boosts and loot info.
Output: Void.
Logic:
Plays a victory sound/visual via ui.js.
Displays the attribute boosts (e.g., "+1 Strength").
Calls endCombatNormally() after a 2-second "Victory Beat" delay.

**6. endCombatNormally()**
Description: The "Clean Exit" function. Resets the game state so navigation and exploration buttons work again.
Input: None.
Output: Void.
Logic:

Sets isCombatActive = false.
Hides the overlay.
Calls get_room.php to refresh the room (this ensures the dead NPC now shows up as "Dead" and its loot appears on the ground).

**7. initiateDefeat()**
Description: Orchestrates the respawn.
Input: None.
Output: Void.
Logic:
Calls reset_game.php (PHP).
Clears combat state and redirects the player to their respawnNodeId.

Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
playerParty
Array
get_room.php
IDs of NPCs currently helping the player.
sttScore
Float
speech.js
The 0.0–1.0 score that determines damage tier.
isCombatActive
Boolean
Global
Logic gate for engine.js and speech.js.
currentEnemy
Object
startCombat()
Reference to the NPC being targeted.

Software Engineer's Compatibility Audit
Stability: By moving the "Attribute Boosts" and "Loot Drops" to the Backend (handle_victory.php), we ensure that if a player's browser crashes during the Victory Beat, their rewards are already saved in the database.
The Ally Logic: By filtering playerParty at the start of combat, we ensure that allies who enter the room after a fight starts don't accidentally join in (preventing logic loops).

---

**process_combat.php**: The Battle Judge

**1. calculate_player_damage($sttScore, $strength)**
Description: Implements the "Grading" logic from your Project Bible.
Input:
$sttScore (Float): The 0.0 to 1.0 similarity score from speech.js.
$strength (Int): The player's current strength attribute.
Output: Integer (Final damage points).

Logic:
Parfait (>= 0.95): $strength * 1.0 (Max Damage).
Bien (>= 0.75): $strength * 0.5 (Half Damage).
Pas compris (< 0.75): 0 (Miss).

**2. calculate_ally_support($playerParty)**
Description: Iterates through the recruited NPCs currently in the room to add their damage to the turn.
Input: $playerParty (Array of NPC IDs).
Output: Integer (Total support damage).
SQL Logic: SELECT strength FROM Npcs WHERE npcId IN (...).
Logic: Sums a percentage of each ally's strength to assist the player.

**3. calculate_npc_counter($npcId, $playerAgility, $playerArmor)**
Description: Calculates the monster's attempt to hit the player back.
Input: $npcId (Int), $playerAgility (Int), $playerArmor (Int).
Output: Integer (Damage dealt to player).
Logic: Uses a "Dodge Check." If Random(1,20) + NPC_Agility > Player_Agility, the hit lands. Damage is then reduced by the playerArmor value.

**4. apply_combat_results($charId, $npcId, $pDamage, $nDamage)**
Description: The primary database writer. It updates health for both entities in one transaction.
Input: $charId (Int), $npcId (Int), $pDamage (Int), $nDamage (Int).
Output: Array (New HP totals).
SQL Logic:
UPDATE Character_NPC_State SET currentHitPoints = currentHitPoints - ? WHERE npcId = ? AND characterId = ?.
UPDATE Characters SET hitPoints = hitPoints - ? WHERE id = ?.

**5. check_death_conditions($playerHP, $npcHP)**
Description: Determines if the battle is over and triggers the appropriate exit logic.
Input: $playerHP (Int), $npcHP (Int).
Output: String ('CONTINUE', 'VICTORY', or 'DEFEAT').
Logic: If npcHP <= 0, it internally calls mark_dead.php. If playerHP <= 0, it flags a reset.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$sttScore
Float
POST (from JS)
The raw pronunciation grade.
$playerDamage
Integer
Calculation
Final damage dealt to the NPC.
$npcDamage
Integer

Calculation
Final damage dealt to the Player.
$isNpcDead
Boolean
Calculation
Flag used to trigger the "Victory" sequence in JS.

Engineering Interaction Audit
Tables it READS from:
Characters: To get player Strength, Agility, and Armor.
Npcs: To get the monster's base stats.
Character_NPC_State: To get the monster's current HP for that player.
Tables it WRITES to:
Characters: Updates the player's health.
Character_NPC_State: Updates the monster's health and isDead status.

Software Engineer's "Bible" Note:
This file handles the NPC Courage mechanic. Inside calculate_npc_counter, we add a check: if NPC_HP < 15% of Max HP, this file returns a flee status instead of a damage value, triggering the NPC's escape to another room.

---

**handle_victory.php**: The Loot and Progression Judge

**1. verify_kill_status($charId, $npcId)**
Description: A security handshake to prevent players from triggering victory rewards manually.
Input: $charId (Int), $npcId (Int).
Output: Boolean.
SQL Logic: SELECT isDead FROM Character_NPC_State WHERE characterId = ? AND npcId = ?.
Verdict: If isDead is not already 1, the script terminates to prevent cheating.

**2. calculate_attribute_boost($npcId)**
Description: Calculates the reward for the player's attributes based on the defeated NPC's power.
Input: $npcId (Int).
Output: Array (Key-value pairs of boosts, e.g., ['strength' => 1, 'experience' => 20]).
Logic: As per your Bible, the boost is a "percentage of the defeated NPC attributes."
Variables: $boostFactor (Float): 0.05 (e.g., 5% of the NPC's strength is added to the player).

**3. transfer_npc_loot($npcId, $nodeId)**
Description: Moves items from the dead NPC's pockets to the floor of the current room.
Input: $npcId (Int), $nodeId (Int).
Output: Int (Number of items dropped).
SQL Logic: UPDATE ItemInstances SET ownerType = 'Room', ownerId = ? WHERE ownerType = 'NPC' AND ownerId = ?.
Bible Connection: Fulfills the requirement: "Victory sequence that drops their items onto the floor."

**4. apply_progression_updates($charId, $boosts, $gold)**
Description: The final write operation to the player's record.
Input: $charId (Int), $boosts (Array), $gold (Decimal).

Output: Boolean.
SQL Logic: UPDATE Characters SET strength = strength + ?, agility = agility + ?, experiencePoints = experiencePoints + ?, gold = gold + ? WHERE id = ?.

**5. generate_victory_json($boosts, $itemsDropped)**
Description: Packages the rewards for the combat.js "Victory Beat" UI.
Input: Array, Int.
Output: JSON String.
Example: {"status": "success", "boosts": {"str": 1, "xp": 20}, "itemsDropped": 2}.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$charId
Integer
Session
The hero receiving the rewards.
$npcId
Integer
POST
The specific entity that was defeated.
$boosts
Array
Calculation
The mathematical gains (STR, AGI, XP).
$nodeId
Integer
Session
Where the loot is physically dropped.

Engineering Interaction Audit
Tables it READS from:
Npcs: To get the base stats of the enemy for boost calculations.
Character_NPC_State: To verify the death and the location.
Tables it WRITES to:
Characters: Permanently increases the player's power and wealth.
ItemInstances: Changes the "owner" of the NPC's items to the Room.
Interaction with Global Utility (init.php):
This file pulls the Attribute Multipliers from init.php so you can adjust game difficulty (how fast players level up) in one place.

Software Engineer's "Bible" Note:
This file is the only place where Attribute Growth happens. By keeping it separate from the combat turn logic, we ensure that loot and level-ups are handled in a single, "clean" database transaction. This prevents the "Double Loot" bug where a player might get rewards twice for one kill.

---

**flee.php**: The Escape Judge

**1. calculate_flee_success($playerAgility, $npcAgility)**
Description: Determines if the player successfully evades the enemy.
Input: $playerAgility (Int), $npcAgility (Int).
Output: Boolean (True if escape succeeds).
Logic: As per your vision, this is a contested check.
Formula: (Player_Agility + Random(1,10)) > (NPC_Agility + Random(1,10)).
Verdict: If the player is much faster than the NPC, they escape easily.

**2. calculate_opportunity_attack($npcStrength, $playerArmor)**
Description: Implements the "33% chance at another attack" rule from your Project Bible.
Input: $npcStrength (Int), $playerArmor (Int).
Output: Integer (Damage dealt during escape).
Logic:
Generates Random(1, 100).
If result <= 33, calculate damage: NPC_Strength - Player_Armor.
Verdict: This adds the "cost" to fleeing that prevents players from abusing the escape button.

**3. identify_escape_route($currentCharId)**
Description: Determines where the player actually goes when they run away.
Input: $currentCharId (Int).
Output: Integer (A valid nodeId).
Logic: It fetches the previous nodeId the player was in before the current one, or picks a random valid TargetID from the current room that is not 0.

**4. apply_flee_results($charId, $newHp, $newNodeId)**
Description: Updates the database to reflect the damage taken and the new location.
Input: $charId (Int), $newHp (Int), $newNodeId (Int).
Output: Boolean.
SQL Logic: UPDATE Characters SET hitPoints = ?, currentLocationID = ? WHERE id = ?.

**5. generate_flee_json($success, $damageTaken, $targetNode)**
Description: The "Plating" function that tells the frontend what happened.
Input: Bool, Int, Int.
Output: JSON String.
Example: {"status": "escaped", "damageTaken": 5, "targetNode": 101}.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$playerAgility
Integer
Characters
Used to calculate escape success.
$npcStrength
Integer

Npcs
The raw power of the monster's parting blow.
$damageDealt
Integer
Calculation
The 33% chance hit reduction in HP.
$targetNode
Integer
Locations
The destination ID for the fleeing player.

Engineering Interaction Audit
Tables it READS from:
Characters: To get current health and agility.
Npcs: To get the monster's agility and strength.
Locations: To find a valid exit for the escape.
Tables it WRITES to:
Characters: Updates the new (lower) HP and the new currentLocationID.
Interaction with combat.js:
When the JS receives the JSON, it triggers a screen shake (if damage was taken) and then forces a ui.displayRoom() call to refresh the view in the new "Safe" location.

Software Engineer's "Bible" Note:
This file ensures that Persistence is maintained. If a player flees and takes 5 damage, that damage is written to the database before they arrive in the next room. This prevents a player from fleeing to "undo" the damage they took during the fight.

---

**process_command.php**: The Action Arbiter

**1. interpret_vocal_intent($heardText, $context)**
Description: Translates a raw French string into a specific game action (e.g., "Open," "Read," "Search").
Input:
$heardText (String): The normalized text from speech.js.
$context (Array): Current room data (NPCs/Items present).
Output: String (The identified actionType).
Logic: Uses a keyword mapping (e.g., "Ouvrir" -> OPEN, "Regarder" -> EXAMINE).

**2. execute_examine_logic($target, $charId)**
Description: Handles "Regarder" or "Chercher" commands. It provides deeper bilingual descriptions or reveals hidden items.
Input: $target (String), $charId (Int).
Output: String (Bilingual response text).
SQL Logic: Queries the Locations or ItemLibrary for an extraDescription field (stored in the extraData JSON).

**3. process_reward_economy($score, $charId)**
Description: Implements the "Speak-to-Earn" mechanic from your Project Bible.
Input: $score (Float): The 0.0–1.0 similarity score.
Output: Decimal (Gold earned).

Logic:
If score >= 0.95 (Parfait), add 0.20 Gold.
If score >= 0.75 (Bien), add 0.10 Gold.
SQL Logic: UPDATE Characters SET gold = gold + ?, speechSuccessCount = speechSuccessCount + 1 WHERE id = ?.

**4. trigger_world_event($action, $nodeId)**
Description: Handles permanent changes to a room based on an action (e.g., unlocking a door).
Input: $action (String), $nodeId (Int).
Output: Boolean.
SQL Logic: Updates flags in the Character_Room_State table (e.g., SET doorUnlocked = 1).

**5. generate_command_response($msg, $updates)**
Description: The "Plating" function that tells the frontend what happened as a result of the speech.
Input: $msg (String), $updates (JSON Object).
Output: JSON String.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$heardText
String
POST
The raw French command spoken.
$sttScore
Float
POST
Used to determine the Gold reward amount.
$goldReward
Decimal
Constant
Standardized reward (0.10 or 0.20) in init.php.
$charId
Integer
Session
Identifies the player receiving the gold/state change.

Engineering Interaction Audit
Tables it READS from:
Locations / ItemLibrary: To find descriptions or hidden flags.
Characters: To check current Gold/Charisma levels.
Tables it WRITES to:
Characters: Increases Gold and speechSuccessCount.
Character_Room_State: Records if a room's puzzle or door has been interacted with.

Interaction with speech.js:
This file is the primary destination for the Exploration STT. It converts "Good French" into "Real Money."

Software Engineer's "Bible" Note:
This file fulfills the Reward Economy requirement: "Successful spoken commands grant a small reward to incentivize speaking practice." By placing the Gold logic here, we ensure that whether the player is opening a chest or talking to a guard, the "Judge" always applies the same consistent reward for correct French.

---

**process_item.php**: The Possession Judge

**1. validate_transfer_request($charId, $instanceId, $action)**
Description: Checks if the item actually exists and if it is in the correct location to be moved (e.g., you can't "Take" an item that isn't in your current room).
Input: $charId (Int), $instanceId (Int), $action (String: 'TAKE' | 'DROP' | 'USE').
Output: Boolean (True if the request is valid).
SQL Logic: SELECT ownerType, ownerId FROM ItemInstances WHERE instanceId = ?.
Verdict: This prevents a player from "stealing" an item from a different room via code injection.

**2. check_encumbrance_limit($charId, $instanceId)**
Description: Calculates if adding this item would exceed the player's capacity.
Input: $charId (Int), $instanceId (Int).
Output: Boolean.
Logic:
Fetches player strength (from Characters).
Calculates maxWeight = strength * STR_TO_WEIGHT_CAP.
Sums current inventory weight + new item weight.
Verdict: Returns false if totalWeight > maxWeight.

**3. execute_take_item($charId, $instanceId)**
Description: Updates the ownership of an item on the ground to the player's pocket.
Input: $charId (Int), $instanceId (Int).
Output: Boolean.
SQL Logic: UPDATE ItemInstances SET ownerType = 'Player', ownerId = ? WHERE instanceId = ?.

**4. execute_drop_item($nodeId, $instanceId)**
Description: Updates the ownership of an item in the player's pocket to the current room floor.
Input: $nodeId (Int), $instanceId (Int).
Output: Boolean.
SQL Logic: UPDATE ItemInstances SET ownerType = 'Room', ownerId = ? WHERE instanceId = ?.

**5. execute_use_consumable($charId, $instanceId)**
Description: Processes items with a "Use" effect (like healing) based on their JSON data.
Input: $charId (Int), $instanceId (Int).
Output: Array (The result, e.g., "Healed 20 HP").
Logic:
Reads extraData JSON (e.g., {"heal": 20, "doses": 1}).
Updates Characters.hitPoints.

Cleanup: If doses reaches 0, it runs DELETE FROM ItemInstances WHERE instanceId = ?.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$instanceId
Integer
POST
The unique ID of the specific item being moved.
$charId
Integer
Session
The ID of the player performing the action.
$weight
Decimal
ItemLibrary
The physical weight of the item.
$extraData
JSON
ItemLibrary
Contains the item's effects (healing, durability).

Engineering Interaction Audit
Tables it READS from:
ItemInstances / ItemLibrary: To identify what is being moved and its weight.
Characters: To verify the player's Strength/Encumbrance.
Tables it WRITES to:
ItemInstances: Changes ownerType and ownerId.
Characters: Updates HP or other stats if an item is "Used."
Interaction with WorldTemplates:
None. This file only manages the "Live" state of items.

Software Engineer's "Bible" Note:
This file implements the Unique Items collection rule. Because it targets the instanceId, it can track a specific "Iron Sword" that has 50% durability without affecting any other swords in the world. It also ensures the Reward Economy stays balanced by preventing players from carrying an infinite amount of loot to sell.

---

**mark_dead.php**: The Status Executioner

**1. verify_lethal_state($charId, $npcId)**
Description: A final safety check to ensure the NPC is actually at 0 HP before marking them as dead.
Input: $charId (Int), $npcId (Int).
Output: Boolean.

SQL Logic: SELECT currentHitPoints FROM Character_NPC_State WHERE characterId = ? AND npcId = ?.
Verdict: This prevents a "Ghost Kill" bug where a network lag might try to kill an NPC that still has health.

**2. execute_death_flag($charId, $npcId)**
Description: The primary database write that kills the NPC for that specific player.
Input: $charId (Int), $npcId (Int).
Output: Boolean.
SQL Logic: UPDATE Character_NPC_State SET isDead = 1, currentHitPoints = 0 WHERE characterId = ? AND npcId = ?.
Bible Connection: Fulfills the requirement: "When NPC die they persist in room as dead and inactive."

**3. deactivate_npc_timers($charId, $npcId)**
Description: If your game eventually uses timed respawns, this function logs the "Time of Death."
Input: $charId (Int), $npcId (Int).
Output: Void.
SQL Logic: UPDATE Character_NPC_State SET lastDeathTime = NOW() WHERE characterId = ? AND npcId = ?.

**4. log_kill_to_history($charId, $npcId)**
Description: (Optional/Future Proofing) Records the kill in a global log for "Kill X Monsters" quests.
Input: $charId (Int), $npcId (Int).
Output: Void.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$charId
Integer
Session
The specific player who achieved the kill.
$npcId
Integer
POST
The ID of the monster being deactivated.
$isDead
TINYINT
NPC_State
The 0/1 flag that tells the UI to hide "Attack" buttons.

Engineering Interaction Audit
Tables it READS from:

Character_NPC_State: To check current HP.
Tables it WRITES to:
Character_NPC_State: Flips the isDead bit to 1.
Interaction with ui.js:

Once this file returns "Success," ui.js knows to stop showing the "💬 Parler" button for this NPC and instead show the item names as "Objets au sol" (loot) via handle_victory.php.

Software Engineer's "Bible" Note:
This file ensures Narrative Persistence. Because we use the Character_NPC_State table, if the player kills the Wolf, leaves the room, and comes back 10 minutes later, the Wolf remains dead. This is because get_room.php will see the isDead = 1 flag and tell the UI: "Don't start combat; this entity is inactive."

---

**reset_game.php**: The State Recovery & Penalty Judge

**1. apply_death_penalties($charId)**
Description: Implements the "10% gold loss" rule and clears any temporary combat status effects.
Input: $charId (Int).
Output: Decimal (The amount of gold deducted).
SQL Logic: UPDATE Characters SET gold = gold * (1 - ?) WHERE id = ?.
Variables: Uses the DEATH_GOLD_PENALTY (0.10) constant from init.php.

**2. respawn_player_at_checkpoint($charId)**
Description: Relocates the player to their last saved "High-Water Mark" location.
Input: $charId (Int).
Output: Int (The respawnNodeId they were moved to).
SQL Logic: UPDATE Characters SET currentLocationID = respawnNodeId, hitPoints = maxHitPoints WHERE id = ?.
Bible Connection: Fulfills the requirement: "When PLAYER dies... he retains all other attribute levels... he respawns at the latest saved point."

**3. initialize_new_world_state($charId) (The "Full Reset" logic)**
Description: Used for a total game restart. It clears the player's unique world state and re-syncs from the blueprints.
Input: $charId (Int).
Output: Boolean.
Logic:
DELETE FROM ItemInstances WHERE ownerType = 'Player' AND ownerId = ?.
DELETE FROM Character_Room_State WHERE characterId = ?.
Calls spawn_items_from_template() to repopulate the world.

**4. spawn_items_from_template($charId)**
Description: The "Blueprint Executioner." It copies rows from WorldTemplates into ItemInstances.
Input: $charId (Int).
Output: Int (Number of items spawned).
SQL Logic: INSERT INTO ItemInstances (itemId, ownerType, ownerId) SELECT itemId, ownerType, ownerId FROM WorldTemplates.
Interaction: Ensures the "Rust Key" is back in Room 101 for the new session.

**5. generate_reset_response($type, $msg)**

Description: Packages the result for ui.js to trigger a page refresh or a "Game Over" screen.
Input: $type (String: 'DEATH' | 'FULL_RESET'), $msg (String).
Output: JSON String.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$charId
Integer
Session
The specific hero being reset.
$goldPenalty
Decimal
Constant
Taken from the 10% rule in init.php.
$respawnNode
Integer
Characters
The destination ID for a death respawn.
$templateId
Integer
WorldTemplates
The source record for world initialization.

Engineering Interaction Audit
Tables it READS from:
Characters: To find the respawnNodeId and current Gold.
WorldTemplates: To find the "Default" locations for items and NPCs.
Tables it WRITES to:
Characters: Updates location, health, and gold.
ItemInstances: Clears old items and inserts new ones during a full reset.
Character_Room_State / Character_NPC_State: Wipes progress for a full reset.
Interaction with ui.js:
Once this file returns "Success," ui.js calls location.reload(). This forces the entire engine to re-fetch the "Truth" from the newly reset database.

Software Engineer's "Bible" Note:
This file handles the "Game Starts Over" logic. By keeping the penalty and respawn logic here, we ensure that dying in combat isn't just a UI change—it's a hard database update. The player loses the gold permanently, and the "Truth" of their location is updated on the server before they even see the respawn screen

---

**content_auditor.php**: The Quality Assurance Judge

**1. audit_navigation_integrity()**

Description: Scans the Locations table for "Orphaned Nodes" and "Broken Links."
Input: None.
Output: Array (List of problematic Node IDs).
SQL Logic:
Broken Links: Checks if any targetID points to a nodeId that does not exist.
Orphans: Checks for nodeId values that are not the target of any other room's compass direction.
Bible Connection: Fulfills the requirement to flag "Broken Links or Orphaned Nodes."

**2. audit_linguistic_holes()**
Description: Scans Locations, ItemLibrary, and Npcs for missing translations.
Input: None.
Output: Array (List of missing pairs).
SQL Logic: SELECT * FROM Locations WHERE textFrench IS NULL OR textEnglish IS NULL.
Logic: It identifies where you have a "Concept" but haven't provided the bilingual content yet.

**3. verify_non_euclidean_overrides()**
Description: Identifies "One-Way Paths" or "Slides" where going North from Room A to Room B, then South from Room B, does not lead back to Room A.
Input: None.
Output: Array (List of one-way coordinates).
Logic: It compares A.northTarget = B against B.southTarget = A. If they don't match, it flags it as an override.
Bible Connection: Fulfills the requirement to flag "Non-Euclidean overrides."

**4. audit_checkpoint_pacing()**
Description: Checks the distance between rooms marked with isCheckpoint = 1.
Input: None.
Output: Array (List of "High-Danger" zones).
Logic: It calculates the "Depth" of nodes from the last checkpoint. If a player is 10 rooms away from a save point, it flags it for a difficulty review.

**5. generate_audit_report_json()**
Description: Bundles all diagnostic arrays into a color-coded report for the Admin Console.
Input: Multiple Arrays.
Output: JSON String.
Example: {"status": "warning", "brokenLinks": [404, 505], "linguisticHoles": 12}.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$orphanNodes
Array (Int)
Calculation
List of rooms that no one can walk into.
$missingPairs
Array (String)
Calculation

IDs of items/rooms missing a French or English name.
$isEuclidean
Boolean
Calculation
Result of the A-to-B-to-A link check.

Engineering Interaction Audit
Tables it READS from:
Locations: The primary target for link and text audits.
ItemLibrary: To ensure items have bilingual names.
Npcs: To ensure greetings and names are translated.
Tables it WRITES to:
None. This is a diagnostic tool; it only reports "The Truth," it does not change it.
Interaction with admin.php:
When you click the "Full Audit" button on the sidebar, engine.js calls this file and passes the JSON to ui.js to highlight rows in red in the Admin Console.

Software Engineer's "Bible" Note:
This file is the "Shield" for your Diamond Flow. By running verify_non_euclidean_overrides, you ensure that your branching paths eventually converge and that one-way "slides" are intentional, not accidental data-entry errors in your TSVs.

---

**navbar.php**: The Navigation & Session Fragment

**1. check_active_page()**
Description: Detects which page the user is currently viewing to apply visual highlighting (e.g., glowing the "PLAY" link if they are in adventure.php).
Input: $_SERVER['PHP_SELF'] (String).
Output: String (CSS class name like "active-nav").
Logic: Compares the filename in the URL against the link targets.

**2. render_dynamic_links($isAdmin, $isLoggedIn)**
Description: The "Gatekeeper" of the UI. It decides which buttons to display based on the session data provided by init.php.
Input:
$isAdmin (Boolean/TINYINT): From the Users table.
$isLoggedIn (Boolean): Based on userId presence in session.
Output: HTML Output (Echoed to the browser).
Logic:
If !$isLoggedIn: Shows only "HOME / LOGIN".
If $isLoggedIn: Shows "ADVENTURE" and "LOGOUT".
If $isAdmin: Adds the "ADMIN DASHBOARD" link.

**3. display_session_stats($characterId)**
Description: Pulls "Quick Stats" into the header (e.g., Player Name and Gold) so the user always has their status visible.
Input: $characterId (Int).
Output: HTML String.
SQL Logic: SELECT characterName, gold FROM Characters WHERE id = ?.
Bible Connection: Provides immediate feedback for the Reward Economy (Gold updates).

**4. inject_nav_style_tokens()**
Description: Ensures the navbar.php uses the global CSS variables like --primary-cyan.
Input: None.
Output: HTML/CSS tags.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$userId
Integer
Session (init.php)
Used to verify if an account is logged in.
$isAdmin
TINYINT (0/1)
Users Table
Unlocks the restricted Admin Dashboard link.
$characterId
Integer
Session (init.php)
Used to fetch name/gold for the HUD display.
$currentPath
String
Server Variable
Used for the "active" link highlighting logic.

Engineering Interaction Audit
Tables it READS from:
Users: To verify admin privileges.
Characters: To display the "Quick Stats" (Gold/Name) in the header.
Tables it WRITES to:
None. This is a UI display fragment.
Interaction with init.php:
Essential Dependency. navbar.php must be included after init.php because it relies on the session variables and the database connection ($conn) to do its logic checks.

Software Engineer's "Bible" Note:
This file ensures the Platform Integrity of your RPG. By having the "Logout" logic and "Admin Access" links in a single file, you prevent a situation where a player finds a "Ghost Link" to the Admin area. It also makes the game feel like a "Web App" rather than a series of disconnected pages.

---

i**ndex.php**: The System Entry Point

**1. initialize_system_access()**

Description: Calls init.php but specifically bypasses the "Auth Check" redirect so the user doesn't get stuck in a loop.
Input: None.
Output: Void.
Logic: Resumes the session to see if a userId already exists from a previous cookie.

**2. render_authentication_forms()**
Description: Displays the "Terminal Login" UI we designed. It provides the input fields for the Users table.
Input: None.
Output: HTML Output.
Bible Connection: Provides the interface for the "One Character per Account" (or the updated multi-character) logic.

**3. fetch_available_characters($userId)**
Description: If the user is logged in, this retrieves their existing heroes from the database.
Input: $userId (Int).
Output: Array of Character objects.
SQL Logic: SELECT id, characterName, currentLocationID, hitPoints FROM Characters WHERE ownerId = (SELECT username FROM Users WHERE userId = ?).
Verdict: This populates the "Character Selection" side of your index page.

**4. display_system_welcome()**
Description: Renders the "Linguistic Protocol" message we drafted.
Input: None.
Output: HTML Output.
Bible Connection: Explains the Bilingual Content and Voice Command Interface to new players before they begin.

**5. handle_error_alerts()**
Description: Displays feedback from auth_worker.php (e.g., "Invalid Password" or "Username Taken").
Input: $_GET['error'] (String).
Output: HTML/CSS Overlay.
Logic: Looks for error codes in the URL and translates them into user-friendly "Terminal Errors" in the .console-box.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$isLoggedIn
Boolean
init.php
Determines whether to show the Login form or the Character list.
$userChars
Array
fetch_available_characters
The list of heroes the user can choose to play as.

$playerName
String
POST
The input for creating a new character.
$REMOTE_ADDR
String
$_SERVER
Displayed in your footer as part of the "Terminal" aesthetic.

Engineering Interaction Audit
Tables it READS from:
Users: To verify if the account exists.
Characters: To list existing characters for selection.
Tables it WRITES to:
None. This file is a UI display. The actual "Writing" is handed off to auth_worker.php via a POST request.
Interaction with auth_worker.php:
When the "Login" button is clicked, index.php sends the credentials to the worker. If successful, the worker redirects back to index.php with a "Success" flag to unlock the Character Selection side.

Software Engineer's "Bible" Note:
This file acts as the State Transition for your "High-Level World Design." It ensures that the player doesn't enter the "Nested Diamond" until they have a valid character ID. It also serves as the first point of the "Reward Economy" by showing the player their current Gold and XP in the character selection screen, motivating them to re-enter the game.

---

**adventure.php**: The Primary Game Viewport

**1. enforce_session_integrity()**
Description: The first line of defense. It utilizes init.php to ensure a characterId is active.
Input: $_SESSION['characterId'] (Int).
Output: Void (Executes a Header Redirect to index.php if missing).
Logic: "If the user has no character selected, they cannot be in the adventure."

**2. load_environment_assets()**
Description: Dynamically injects the necessary CSS and JS files into the HTML <head> and <body>.
Input: None.
Output: HTML Link/Script Tags.
Bible Connection: Ensures the Modular Architecture is maintained by loading speech.js and combat.js separately.

**3. initialize_client_state()**
Description: Passes critical PHP session variables into a small JavaScript block so the frontend knows who is playing.
Input: $_SESSION['characterId'] (Int).
Output: Inline JS Variable Declaration.
Example: <script>const activeCharId = 101;</script>

**4. render_interface_grid()**
Description: Builds the 2-column layout defined in your adventure-base.css.
Input: None.
Output: HTML Structure.
Internal IDs:
id="story-box": Target for ui.js room descriptions.
id="stats-panel": Target for player attributes.
id="compass-grid": Target for the 8-point movement buttons.

**5. include_modular_fragments()**
Description: Injects the shared UI components.
Input: None.
Output: HTML/PHP Output.
Files Included:
navbar.php: For consistent site navigation.
combat_overlay.php: (Hidden by default) for the battle arena.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin
Purpose
$characterId
Integer
Session
The unique ID used to fetch the world "Truth."
$isEnglishMode
Boolean
Cookie/Session
Determines if the page starts in FR or EN.
$currentTheme
String
User Prefs
Used to swap CSS files if a "High-Contrast" mode is added.

Engineering Interaction Audit
Tables it READS from:
Characters: To verify the character exists and is currently active.
Tables it WRITES to:
None. This is a layout container.
Interaction with engine.js:
As soon as adventure.php finishes loading in the browser, it triggers engine.initGame(), which starts the cycle of calling get_room.php and get_player.php.

Software Engineer's "Bible" Note:
This file is the "Chassis" for your Hybrid Travel and Dual-Command Interface. It holds the placeholders for the "Return to Base" shortcut and the "Speech-to-Text" microphone button.

Because it is a .php file, it ensures that your "Bilingual Content" is handled securely; the user never sees the logic behind how the French text is pulled—they only see the result.

---

**admin.php**: The System Diagnostic Command Center

**1. verify_admin_access()**
Description: A strict security gate that checks the Users table for the isAdmin flag.
Input: $_SESSION['userId'] (Int).
Output: Void (Executes a Header Redirect to adventure.php or index.php if unauthorized).
Logic: "If a player manually types admin.php into the URL bar, the system kicks them out unless their isAdmin bit is 1."

**2. load_diagnostic_assets()**
Description: Specifically injects admin-base.css to trigger the "System Override" visual theme (the solid cyan background).
Input: None.
Output: HTML Link Tags.
Logic: It overrides the adventure-base.css defaults to create the high-contrast dashboard look.

**3. initialize_auditor_hooks()**
Description: Sets up the JavaScript event listeners for the sidebar buttons (Audit Database, Path Audit, Full Audit).
Input: None.
Output: JS Event Listeners.
Logic: Links the physical buttons to calls for content_auditor.php.

**4. render_diagnostic_console()**
Description: Builds the "Black Console" area where the results of the audits are printed.
Input: None.
Output: HTML Structure (class="admin-console").
Bible Connection: This is the terminal window where "Linguistic Holes" and "Broken Links" are reported in red text.

**5. fetch_system_metrics()**
Description: Gathers high-level "Truth" data about the game's scale.
Input: None.
Output: Array.
SQL Logic:
SELECT COUNT(*) FROM Locations
SELECT COUNT(*) FROM Users
SELECT COUNT(*) FROM ItemLibrary
Verdict: Displays the "Total Rooms," "Total Players," and "Total Items" in the overview section.

Detailed Variable & Data Type Mapping

Variable Name
Data Type
Origin

Purpose
$isAdmin
Boolean
Users Table
The master key that unlocks this page.
$auditResults
JSON Object
content_auditor.php
The raw data that gets color-coded for the console.
$totalNodes
Integer
Locations Table
Used to verify the scope of the world map.

Engineering Interaction Audit
Tables it READS from:
Users: To verify admin status.
Locations / Npcs / ItemLibrary: To count and audit the world data.
Tables it WRITES to:
None. As an auditor, it should not change data. (Note: You might add "Write" ability later for a CMS, but for now, it is a "Read-Only" Diagnostic tool).
Interaction with content_auditor.php:
admin.php is the Display; content_auditor.php is the Engine. When the admin clicks "Path Audit," the page sends a request to the auditor and prints the returned "Orphaned Nodes" list.

Software Engineer's "Bible" Note:
This file is the host for your Visualizer. By using the data from the Locations table (mapX, mapY, mapZ), this page can generate the color-coded grid map to show the "Health" of your Diamond Flow. If a region is missing "Checkpoints," the Visualizer can highlight those rooms in Red to alert the developer.